Practical Application of Agile PM Principles

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

This slide contains material that is copied from the book Agile Project Manager_v2_handbook by Agile Business Consortium. All rights reserved. No part of this material may be reproduced or transmitted in any form or by any means without the prior written permission of the copyright owner. This material is used here for educational purposes under the doctrine of fair use, with the intention to support learning and teaching.

For more information, please refer to the original source:
Book Title: Agile Project Manager_v2_handbook
Author: Agile Business Consortium
Publisher: Buckland Media Group Limited
Year of Publication: 2017
Please consult the book for further details and complete content.

innovative • entrepreneurial • global

Outline

- DSDM Practice : MoSCoW Prioritization
  - Introduction
  - The MoSCoW Rules
  - MoSCow relating to a Specific Timeframe

- DSDM Practice : Timeboxing
  - Timeboxing Options
  - The Daily Stand Up (Scrum)
  - Timeboxes - The Wider Context

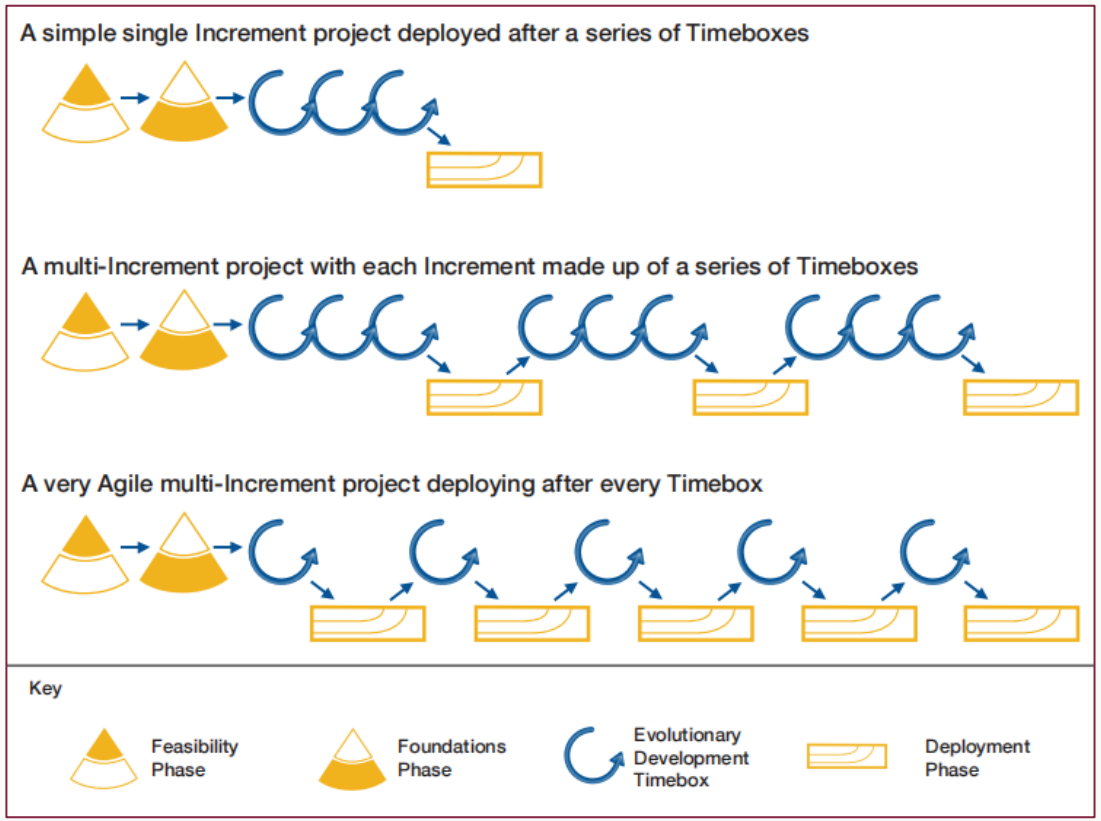
DSDM Practice : MoSCoW Prioritisation

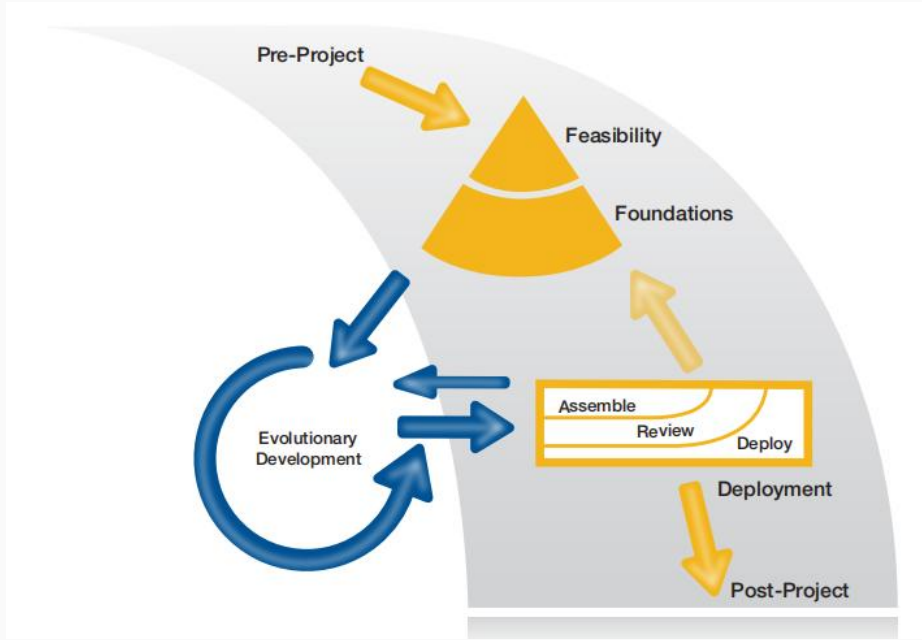
UTM
UNIVERSITI TEKNOLOGI MALAYSIA

innovative • entrepreneurial • global

Recap - Configuring the Process - 3 lifecycle examples

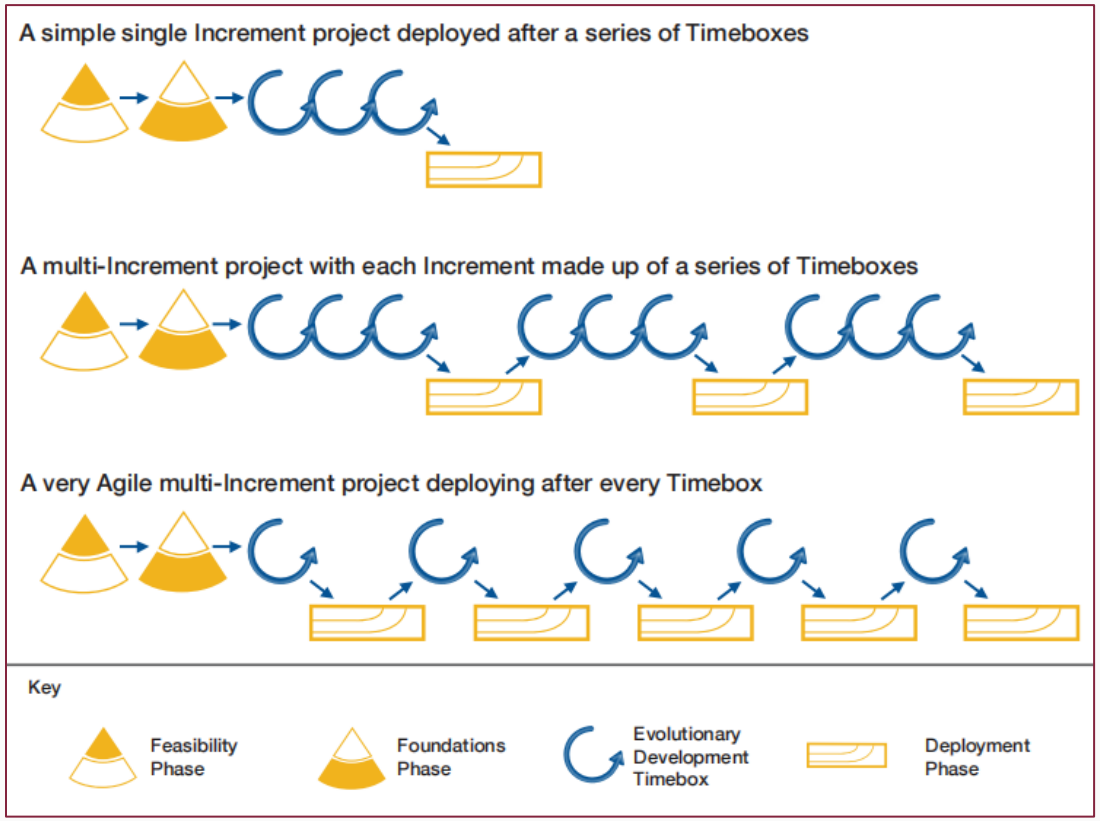
- Solution built and deployed incrementally from a firm foundation
- Ensure value delivered early and delivered frequently





3 Options lifecycle

DSDM Lifecycle Option	Description / Implementation Approach	Example of How Phases Are Organized
1. Single-Increment Project	• The simplest form of DSDM implementation.• The project goes through Feasibility and Foundations once, then proceeds through several Timeboxes (iterations) in the Evolutionary Development Phase before a single Deployment at the end.• Suitable for smaller, well-defined projects with stable requirements.	**Sequence:**Feasibility → Foundations → (Multiple Evolutionary Development Timeboxes) → Single Deployment → Post-Project.**Example:** A small internal HR application built and deployed once after full iteration cycles.
2. Multi-Increment Project	• The project is divided into multiple Increments, each made up of several Timeboxes.• Each Increment represents a partial but usable version of the overall solution.• After each Increment, a Deployment occurs, followed by the next Increment cycle.• Suitable for medium-to-large projects where early releases deliver business value progressively.	**Sequence:**Feasibility → Foundations → Increment 1 (several Timeboxes → Deployment) → Increment 2 (several Timeboxes → Deployment) → Increment 3 (several Timeboxes → Deployment) → Post-Project.**Example:** An e-learning platform where each Increment adds a new module (e.g., user management, course upload, quiz engine).
3. Very Agile Multi-Increment Project	• The most agile configuration of DSDM.• Deployment occurs after every Timebox instead of after a series of Timeboxes.• Ensures very frequent delivery, tight feedback loops, and rapid adaptation to change.• Suitable for dynamic environments or continuous delivery setups.	**Sequence:**Feasibility → Foundations → (Timebox → Deployment → Timebox → Deployment ...) → Post-Project.**Example:** A SaaS web application that releases updates or features after every sprint (e.g., new dashboard, bug fixes, small enhancements).



DSDM Phases, Key Products, and Roles Involved (with Evolutionary Phase)

DSDM Phase	Key DSDM Products (Deliverables)	Roles & Their Description / Focus in This Phase
1. Pre-Project	• Terms of Reference (ToR)	**Business Sponsor** – Approves project concept and initial funding. **Business Visionary** – Outlines desired outcomes and business drivers. **Project Manager** – Plans initial setup, identifies stakeholders. **Technical Coordinator** – Advises on initial feasibility and technology suitability.
2. Feasibility Phase	• Feasibility Assessment • Feasibility Prototype (optional)	**Business Analyst** – Defines outline requirements and assesses viability. **Technical Coordinator** – Evaluates technical feasibility and constraints. **Business Visionary** – Confirms business alignment. **Project Manager** – Coordinates feasibility study, estimates scope, time, and risk.
3. Foundations Phase	• Business Case • Business Vision • Prioritised Requirements List (PRL) • Solution Architecture Definition (SAD) • Management Approach Definition (MAD) • Development Approach Definition (DAD) • Delivery Plan	**Business Visionary** – Defines vision and validates alignment with goals. **Business Sponsor** – Approves the Business Case and baseline scope. **Project Manager** – Leads planning, governance, and ensures consensus. **Business Analyst** – Develops PRL and ensures traceability. **Technical Coordinator** – Designs solution architecture and standards. **Business Ambassador** – Validates business priorities and feasibility.

DSDM Phases, Key Products, and Roles Involved (with Evolutionary Phase)

4. Evolutionary Phase *(combines Exploration + Engineering)*	• Timebox Plan(s) • Evolving Solution • Solution Increment(s) • Daily/Timebox Review Records • Release Plan	**Team Leader** – Facilitates team collaboration and delivery focus. **Business Ambassador** – Provides business input, validates requirements, and accepts deliverables. **Business Analyst** – Supports communication between business and developers, manages evolving requirements. **Solution Developer** – Designs, builds, and integrates solution components. **Solution Tester** – Verifies each increment within the timebox. **Technical Coordinator** – Ensures architecture integrity and standards. **Project Manager** – Monitors progress, manages risks, and supports replanning as needed.
5. Deployment Phase	• Deployed Solution • Benefits Assessment • Project Review Report	**Project Manager** – Oversees release and handover to business. **Business Visionary** – Confirms delivery meets business objectives. **Business Ambassador** – Supports acceptance and user transition. **Technical Coordinator** – Oversees deployment and ensures operational readiness. **Solution Developer/Tester** – Provides release and technical support.
6. Post-Project	• Benefits Realisation Review	**Business Sponsor** – Confirms achievement of planned benefits. **Business Visionary** – Measures long-term impact and value. **Project Manager** – Conducts post-implementation review. **Business Analyst** – Documents lessons learned and improvement opportunities.

DSDM Phases — Typical Challenges and How Products Solve

DSDM Phase	Possible Key Challenges	Related Product(s)	How the Product Solves / Clarifies the Challenge
1. Pre-Project	• Lack of clarity on project purpose or business justification. • Unclear sponsorship or funding commitment. • Misalignment of expectations between stakeholders.	**Terms of Reference (ToR)**	Summarises the project rationale, objectives, and expected business benefits. Establishes initial scope and confirms sponsor commitment, ensuring everyone starts with a common understanding.
2. Feasibility Phase	• Uncertainty whether the project is viable or worth pursuing. • Lack of understanding of risks, constraints, or dependencies. • Misalignment between technical feasibility and business goals.	**Feasibility Assessment** **Feasibility Prototype (optional)**	Feasibility Assessment analyses business, technical, and financial viability. Prototype provides tangible proof of concept to validate assumptions early and avoid waste.
3. Foundations Phase	• Misalignment between business and technical understanding. • Undefined scope or unclear priorities. • Unclear governance and approval process for changes. • Architecture not linked to business goals.	**Business Case** – clarifies justification and benefits. **Business Vision** – defines end-goal and desired outcomes. **Prioritised Requirements List (PRL)** – lists and ranks requirements (MoSCoW). **Solution Architecture Definition (SAD)** – aligns design with business need. **Management Approach Definition (MAD)** – defines roles, decision process, and responsibilities. **Development Approach Definition (DAD)** – outlines technical and build approach. **Delivery Plan** – sets high-level delivery roadmap.	Together, these products establish a clear baseline for scope, roles, governance, and architecture — removing ambiguity before development begins. They ensure both business and technical teams share the same foundation.

DSDM Phases — Typical Challenges and How Products Solve

4. Evolutionary Development Phase *(combining Exploration & Engineering)*	• Ambiguity in interpreting requirements. • Miscommunication between business and developers. • Frequent requirement changes without control. • Limited user feedback or late validation. • Integration or performance issues detected late. • Incomplete test coverage or lack of deployment readiness.	**Timebox Plan** – defines short-term deliverables and priorities. **Evolving Solution** – continuously tested and refined prototype. **Solution Increment** – provides deployable pieces of the system. **Release Plan** – coordinates delivery to business. **Daily/Timebox Review Records** – document feedback and agreement.	These products ensure iterative, collaborative progress. Regular reviews and MoSCoW prioritisation allow flexibility and early feedback. Continuous testing and integration reduce risk of late defects, while the Release Plan ensures readiness for deployment.
5. Deployment Phase	• Resistance to change during rollout. • Unclear handover or ownership. • Benefits not measured or reported. • Lessons not captured for improvement.	**Deployed Solution** – formal delivery and acceptance. **Benefits Assessment** – measures outcomes against Business Case. **Project Review Report** – documents lessons learned.	Ensures solution acceptance, verifies expected value delivery, and captures organisational learning for future projects.
6. Post-Project	• Benefits not tracked after delivery. • Lack of accountability for realised value. • Missed opportunities for process improvement.	**Benefits Realisation Review**	Confirms actual vs. planned benefits, validates ROI, and records lessons to guide future initiatives. Keeps sponsors accountable for achieving long-term outcomes.

DSDM Practice - MoSCoW Prioritisation

MoSCoW Prioritisation is a technique used to rank requirements by importance in Agile projects. It classifies them into four categories:

Must have = **Minimum Usable SubseT (Guaranteed)**
Without this requirement . no point on this date or not legal or unsafe or not a viable solution

Should have — Important but not vital
May be painful to leave out but solution still viable
May need workaround, which may be temporary

Could have — Wanted or desirable but less important
Less impact if left out (compared with a Should)

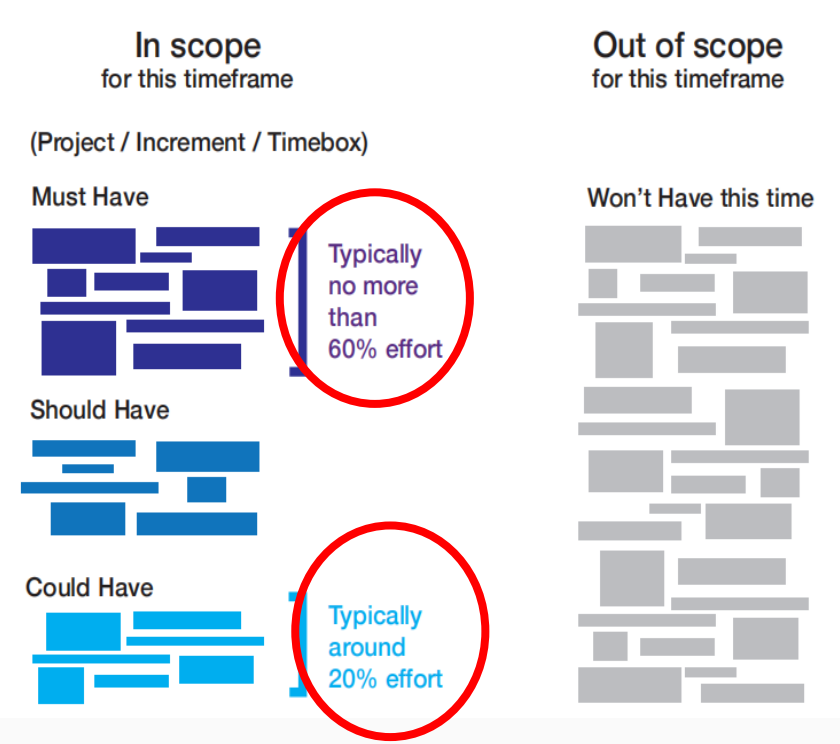
Won't have this time
Will not be delivered in this timeframe



MoSCoW Relating to a Specific Timeframe

This diagram shows how **MoSCoW prioritisation** allocates effort within a project or timebox:

- **Must Have (≤60% effort):** Critical features that must be delivered for the solution to work.
- **Should Have:** Important but not vital; can be postponed if needed.
- **Could Have (~20% effort):** Nice-to-have features that provide added value if time allows.
- **Won't Have this time:** Agreed as out of scope for the current timeframe, possibly for future releases.



It ensures focus on essentials while allowing flexibility for change and risk management.

Example: MoSCoW Effort Allocation in DSDM (Foundations Phase)

MoSCoW Category	Description	% of Total Effort (Typical in DSDM)	No. of Requirements (out of 100)	Estimated Hours Required	Estimated Days Required (at 24h/day)
Must Have	Essential for the solution to work; cannot drop	60%	60	0.60 × 960 = **576 hours**	576 ÷ 24 = **24 days**
Should Have	Important but not critical; may defer if time limited	20%	20	0.20 × 960 = **192 hours**	192 ÷ 24 = **8 days**
Could Have	Nice-to-have; used to absorb time variation	20%	20	0.20 × 960 = **192 hours**	192 ÷ 24 = **8 days**
Won't Have (this time)	Out of scope for this timeframe	–	0	–	–

Assume, 960 man-hours needed for this project, calculate how many days is needed?

Making MoSCoW Work

- Agree up-front how priorities will work

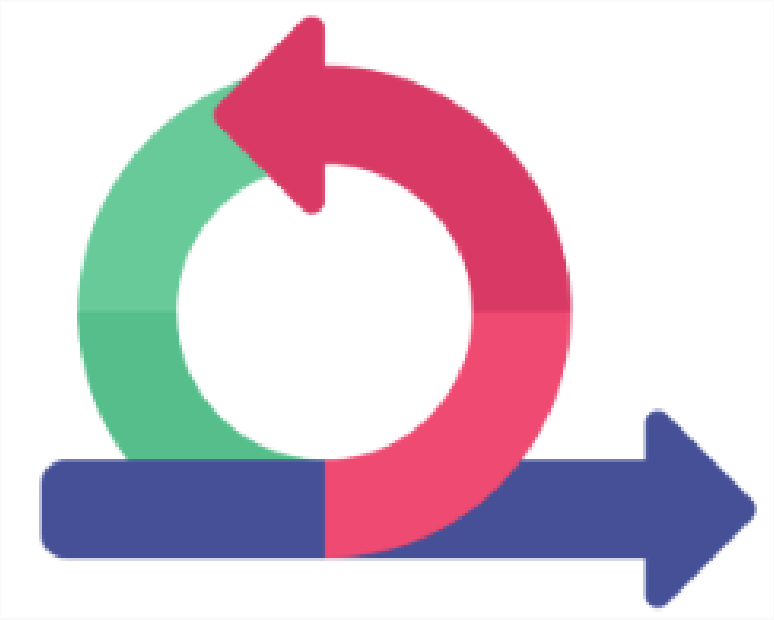
  - Must Haves non-negotiable

  - Should Haves and Could Haves are subjective

  - Discussion facilitated by the Project Manager and Business Analyst

- Majority of prioritisation starts at Foundations but priorities continually reviewed - they change over time
- When new requirements added, check impact

  - On project and on balance of MoSCoW effort



Making MoSCoW Work

- At end of Increment, all requirements not met are re-prioritised according to the need of the next Increment
- Use MoSCoW to manage business expectations

  - High % of Should Haves and Could Haves provide optimum contingency to ensure delivery of Must Haves
- PM responsible for ensuring Delivery Plan is realistic and achievable

More Example

Table 1: MoSCoW Allocation Across 4 Timeboxes (All Completed by TB4)

Timebox	Requirements Planned	Must Have	Should Have	Could Have	Key Notes
Foundations (Baseline PRL)	100	60	20	20	Priorities agreed and effort estimated using MoSCoW (960 hrs total).
Timebox 1	25	15	6	4	Core setup; focus on essential features to establish base functionality.
Timebox 2	25	12	8	5	Reprioritisation occurs — some "Should" items moved to TB3 due to time limits.
Timebox 3	25	10	9	6	Balance adjusted — "Could" items dropped temporarily to ensure "Should" items progress.
Timebox 4	25	8	10	7	New requirements added and reprioritised; all remaining items completed for deployment.
Total (End of TB4)	100	45 + 15 (TB4) = 60	33 + 7 (TB4) = 40	15 + 5 (TB4) = 20	**All requirements completed; system ready for release.**

More Example, helpful to plan TB

Table: Estimated Working Days per MoSCoW Category (within 78 days total)

Timebox	Requirements (Total)	Must Have	Est. Days (Must)	Should Have	Est. Days (Should)	Could Have	Est. Days (Could)	Total TB Days	Key Focus
TB1	25	15	15 days	6	3 days	4	2 days	20 days	Core setup; essential base
TB2	25	12	10 days	8	6 days	5	4 days	20 days	Reprioritisation; some "Should" deferred
TB3	25	10	8 days	9	7 days	6	5 days	20 days	Balance; ensure "Should" items progress
TB4	25	8	7 days	10	8 days	7	5 days	20 days	Final refinement; all completed
Total (End of TB4)	100	45 + 15 = 60	40 days	33 + 7 = 40	24 days	15 + 5 = 20	16 days	≈ 78 days	System ready for release

This simulation assumes one person-day of effort. If two developers are assigned, can the project duration be shortened?

More Example

Table 2: Example of Reprioritisation Scenarios Between Timeboxes

Scenario	Description	Example from HIP 2.0 Project Context	Result / Action Taken
1. Review priorities – some "Should" deferred	When schedule tight, non-critical "Should" features postponed to next TB.	"Automated supplier alert system" deferred from TB2 → TB3.	Time saved ensures completion of Must Have "Supplier registration validation".
2. Rebalance effort – Could Haves used for flexibility	If progress faster than expected, use remaining time to include some "Could" features.	"Dashboard colour themes" (Could Have) added back in TB3.	Increases user satisfaction without impacting deadlines.
3. New requirements reprioritised – final refinements	New requests or feedback introduced late; reprioritised against existing MoSCoW list.	New "Halal certificate renewal reminder" added in TB4 after pilot feedback.	Replaced low-value Could Have "Export summary chart" in final delivery.

Prioritisation - Who Decides?

- Are all Must Haves non-negotiable?
  - Must Have = "Deliver this or we cancel the project
  - PM or Business Analyst may challenge less obvious Must Haves
  - Can this requirement be broken down further?
    - A high-level Must may contain a mix of Musts, Shoulds, Coulds, Won'ts
- Business Visionary and Ambassador have final say
- The Business Sponsor perspective
  - Expects all Must Haves
  - Typically also expects most / all Should Haves to be delivered
  - DSDM's recommended % split gives at least 20% "contingency" for Could Haves



Note: 10% contingency is the normal contingency on traditional projects

Assigning Priorities - AgilePM Top Tips

- Ensure business roles understand why and how DSDM priorities requirements
- Consider starting all requirements as Won't Have, then justify why priority is higher
- Question Must Haves

  - "If on night before Deployment, this requirement is not ready, would you cancel project?"

  - "Is there a workaround, even if it is painful?" If Yes, then it is not a Must
- Ask why this requirement is needed? For Project? For Increment?

Assigning Priorities - AgilePM Top Tips

- Is this requirement dependent on completion of other requirements?

  - A Must cannot depend on a Should Have, Could Have or Won't Have

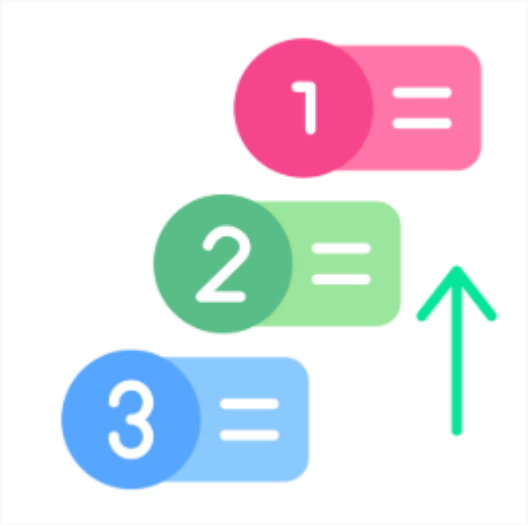
- Allow for different priorities of acceptance criteria
- Can this requirement be decomposed?

  Decomposed elements may have different priorities

- Tie requirements to a project objective :

  If objective not a Must, then requirement cannot be a Must

- MoSCoW helps prevent effort spent on "interesting" rather than "important" things



More example

Table: AgilePM Top Tips for Assigning Priorities (MoSCoW in DSDM)

AgilePM Tip	Explanation	Example (HIP 2.0 Project)
1. Ensure business roles understand why and how DSDM prioritises requirements	Everyone must agree on the importance of prioritisation and its impact on delivery. PM and BA facilitate this understanding.	The Business Ambassador and Visionary agree that only "Must Have" features go live first to ensure the platform functions correctly.
2. Start all requirements as 'Won't Have', then justify why they're higher	Encourages strong reasoning before upgrading an item to Must/Should/Could; prevents overloading the plan.	Start all 100 HIP requirements as Won't Have; "User registration" is justified as Must because system cannot function without it.
3. Question all Must Haves	Test each "Must" using these questions: • *If not delivered, would we cancel the project?* • *Is there a workaround?* If yes, it's not a Must.	"Payment gateway integration" has a manual workaround via bank transfer — hence reclassified from Must to Should.
4. Ask why the requirement is needed (Project or Increment?)	Clarifies timing and purpose — some features may be more relevant later.	"Supplier performance analytics" needed only in Phase 2 → deferred to next increment.
5. Check dependency on other requirements	A Must cannot depend on a Should/Could/Won't Have. Dependencies must align with priorities.	"Report export" (Should) depends on "Dashboard module" (Must), so dashboard must be completed first.
6. Allow for different priorities in acceptance criteria	A requirement can have sub-parts with varied priorities.	"User login" → Must: basic authentication; Should: 2FA; Could: biometric login.

More example

7. Decompose high-level requirements	Break large Musts into smaller items which can have different priorities.	"Supplier management" split into "Add supplier" (Must), "Edit supplier" (Should), "Archive supplier" (Could).
8. Tie requirements to project objectives	If a requirement doesn't directly support a project goal, it shouldn't be a Must.	"Halal certificate validation" directly supports HIP 2.0's compliance goal → Must Have.
9. Use MoSCoW to manage business expectations	Reminds stakeholders that not everything can be delivered immediately; supports realistic planning.	Business Sponsor expects 100% delivery but PM reminds that Could Haves act as contingency.
10. MoSCoW helps avoid spending effort on 'interesting' instead of 'important' items	Keeps focus on business value, not personal preferences.	"Animated homepage banner" dropped (interesting) to focus on "Audit report generator" (important).

DSDM Practice : Timeboxing

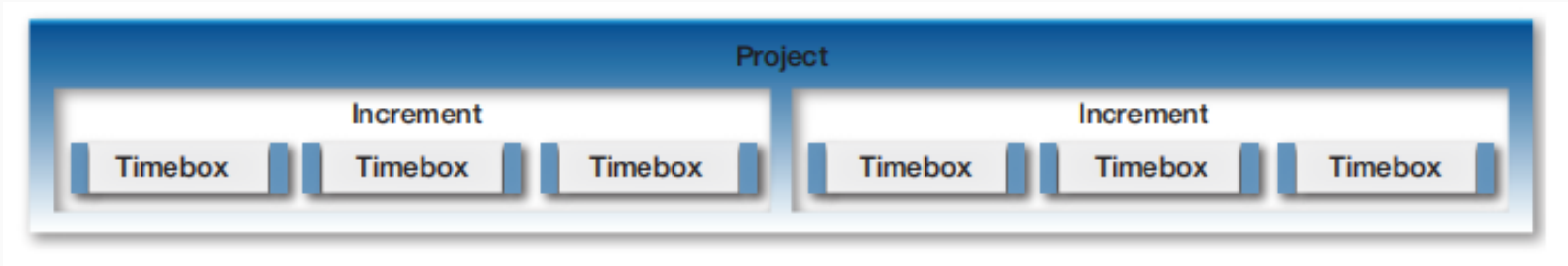
UTM
UNIVERSITI TEKNOLOGI MALAYSIA

innovative • entrepreneurial • global

Timeboxes - How Long?

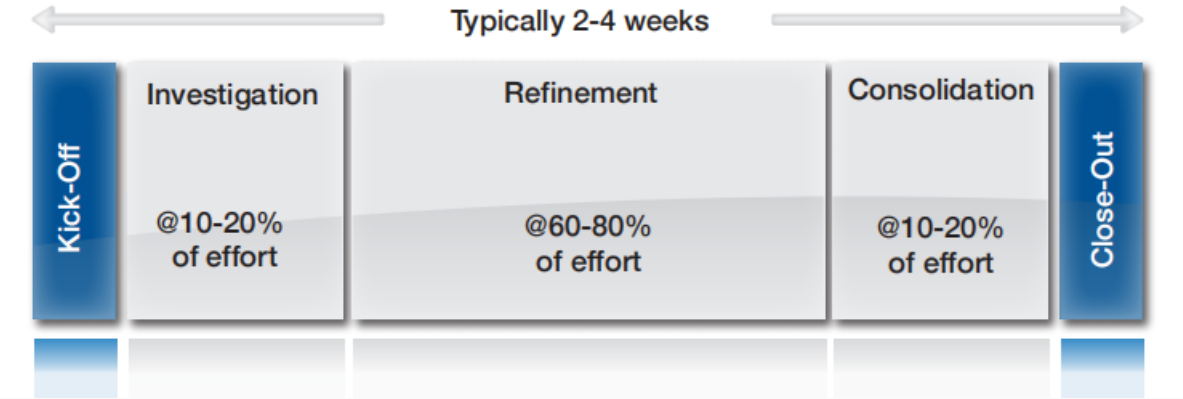
During Foundations, Agile PM helps Solution Development Team agree preferred length of Timebox

- Short enough to keep team focused
- Long enough to achieve something complete and meaningful
- Usually between 2 and 4 weeks
- Longer Timeboxes should be avoided unless really necessary



Timeboxing - Which style?

- DSDM Structured Timebox



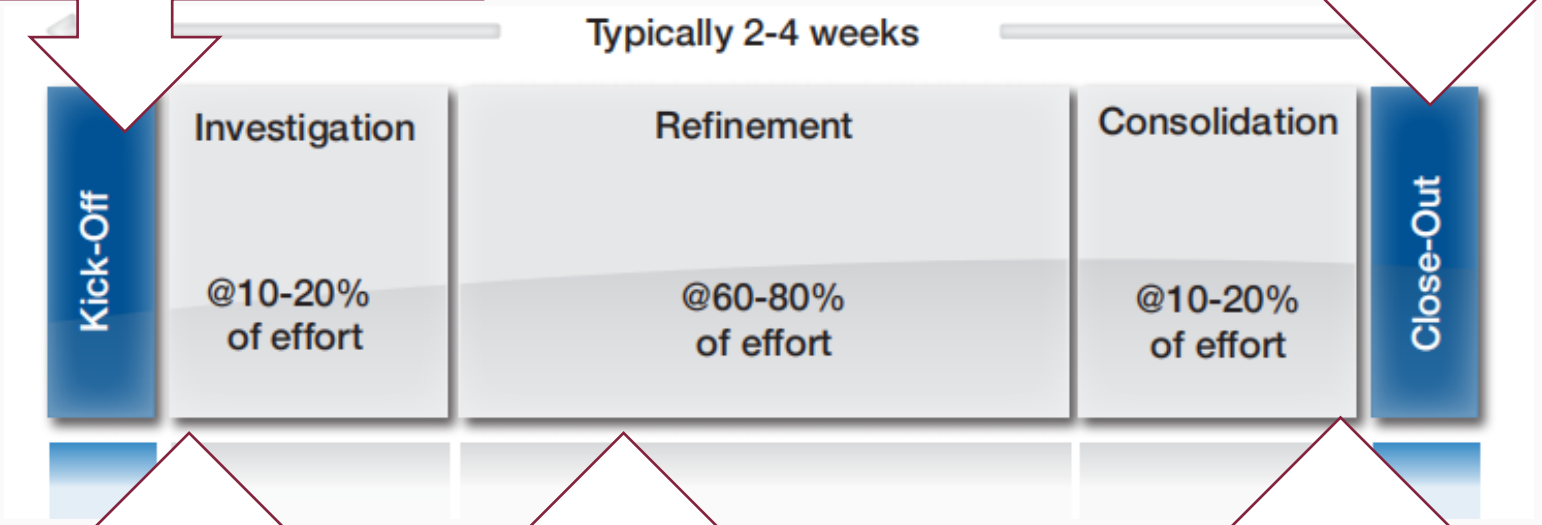
- Free Format Timebox



A DSDM Structured Timebox

- Check TB objectives in Delivery Plan still feasible
- Agree acceptance criteria, where possible
- Review availability of SDT members
- Highlight known dependencies

- Sign-off what has been delivered
- Assess impact of what has not been "done"
- Build on good experience
- Avoid repeating same mistakes
- Define issues to be resolved for next TB



- Understand detail and priorities of work to be done
- Review dependencies, Timebox Plan and risks

- Work on the Solution in line with agreed MoSCoW priorities
- Review at end of Refinement (work should be nearly ready)

- Complete actions from Refinement review
- Complete final testing
- Ensure organisational or project standards have been met, with any final quality checks
- Review at end of Consolidation, ensuring TB objectives are met.
- Any products not meeting acceptance criteria deemed as "Not Delivered"

Timeboxing - Which style?



- Reflects style used by other popular Agile approaches e.g. Scrum Sprint
- Used when DSDM Structured Timebox not possible or not helpful
- Starts with Kick-off and finishes with Close-out
- In between, any number of formal or informal review points
- SDT pick up one or more products or requirements/User Stories and evolve these iteratively until completed and meet agreed acceptance criteria
- SDT then pick up the next product or requirement/User Story and repeat the process
- Free format style relies on Business Ambassador being available consistently to review and provide feedback on ongoing basis

Timeboxing - Agile PM Top Tips

- Ask "How soon can Tester start testing?" and "When is the earliest we can complete a User Story?" Early completion (within a few days) demonstrates tangible progress
- Believe in the TB : encourage SDT to finish on time by prioritising work to be done
- Never let TB "overrun"
- Insist works stops at agreed time and work out what is "done"
- Even if TB has correct balance of priorities, individual skills and capabilities of SDT may still make achieving objectives difficult / impossible

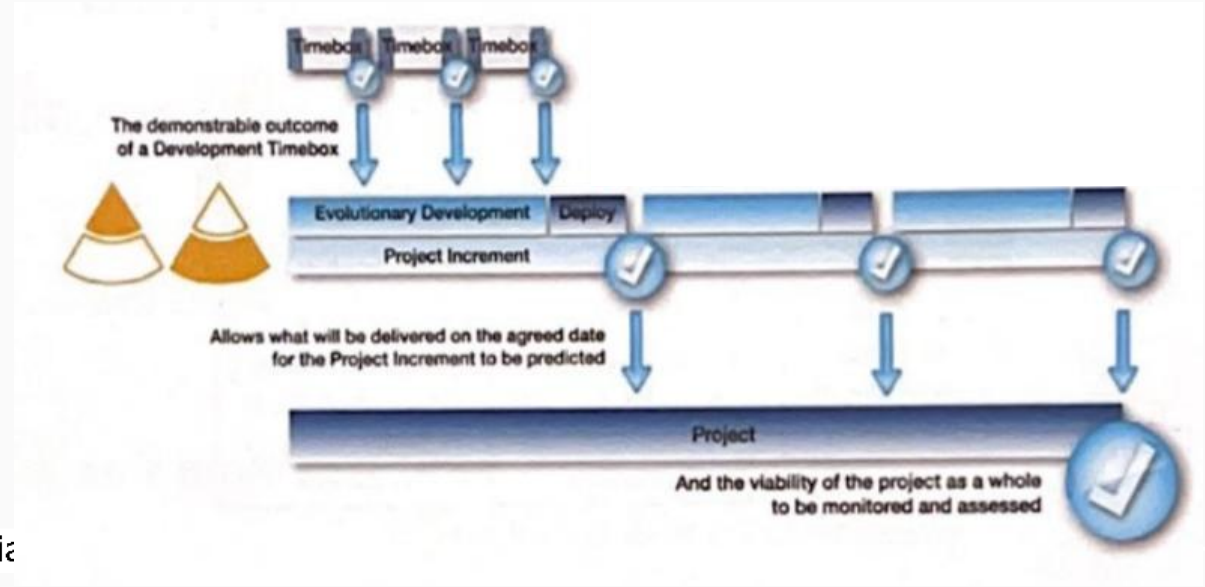
Timeboxing - Agile PM Top Tips

- Check TB progress by observing daily Stand-Ups and enabling SDT to react quickly to problems raised
- Build environment of trust and no-blame, to ensure SDT are open about progress
- Ensure TB Kick-off involves right people
- Ensure acceptance criteria for successful TB completion are clear at Kick-off
- Ensure key risks affecting TB are identified at Kick-off, and monitored during TB
- Ensure Close-out performs a Retrospective to feed learnings into next TBs



Timeboxing - Which style?

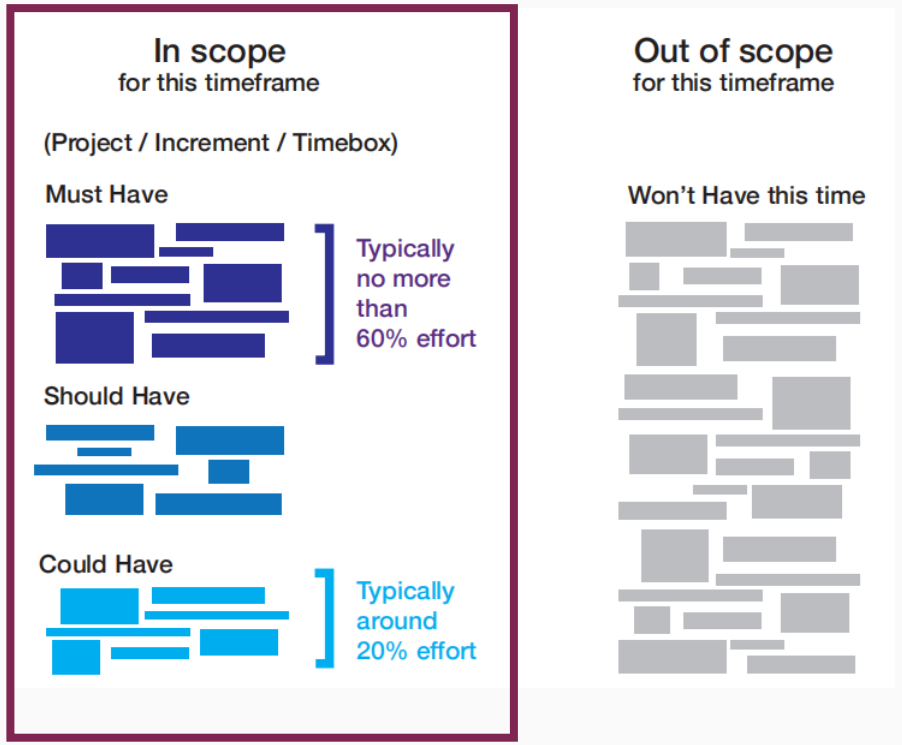
- Combining Timeboxing with MoSCoW enables consistent and predictable on-time delivery of projects
- Project and Increment delivery founded on delivery of Development TBs
- Development Timeboxes:
  - Provide control at lowest level - on-going validation of Delivery Plan
  - Provide clear indication of project health
  - Prove estimating is working (or quickly highlight if not working)
  - Prove SDT is working effectively (or quickly highlight if not effective)
- Confidence feeds upwards to Increment and Project levels



MoSCoW - Integral Part of Planning Process

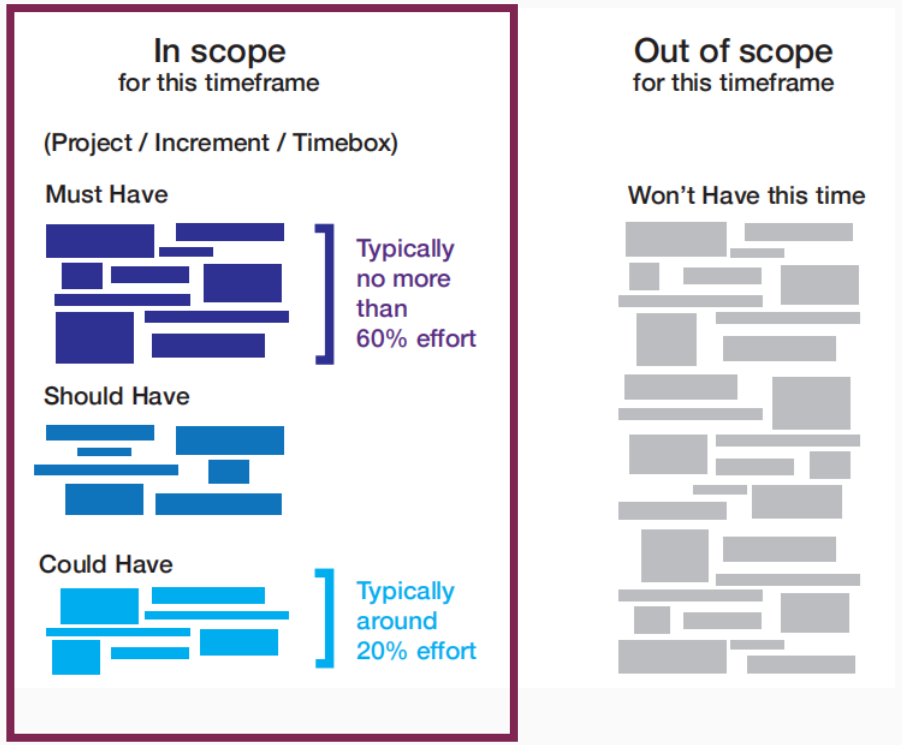
Getting sensible MoSCoW balance at Project and Increment levels achieved through collaboration between whole team

- Business roles work together
  - To ensure priorities reflect true business needs
  - To group requirements to provide effective and valuable solution to meet business deadlines
- Solution / Technical roles work together
  - To ensure plans to deliver these priorities are realistic, achievable reflect any technical constraints and are being planned in most efficient way



MoSCoW - Integral Part of Planning Process

- Business Analyst
    - Considers links between requirements and availability of information
        - Helps to identify viable groups of requirements
        - Agile Project Manager
        - Ensures balance of priorities reflects level of known risks on this specific project



The Daily Stand-up and the AgilePM

- Opportunity for Solution Development Team
  - To share information
  - To carry out day-day re-planning and re-organising
- This is in addition to daily ongoing informal SDT communication
- Agile PM regularly attends, primarily as observer
  - To understand progress and pick up on escalated issues
  - But always remember Stand-up is run by SDT and for SDT
    - They are not "Reporting" to PM
- Visionary and Technical Coordinator also observe to keep in touch with progress
  - Visionary is also providing visible ongoing support
  - Technical Coordinator is keeping up with technical decisions and picking up escalated technical issues

The Daily Stand-up and the AgilePM

- A key part of all Timeboxes
- Takes place same time, same place every day
- SDT share information to enable any day-to-day replanning and reorganising
- Simple format
  1. What I have done since the last stand-up to help achieve the Timebox objectives?
  2. What I will be doing until the next stand-up to help achieve the Timebox objectives?
  3. What problems, risks or issues (blockers) do I have that will prevent me or the team achieving the Timebox objectives?
- As a guide, allow 2 minutes per person + 2 minutes

Empowered Teams and Change in Timeboxes

- SDT must be appropriately empowered to handle any change falling within agreed TB objectives without formal change control
  - i.e. without needing to go outside SDT
- As a guide, following scenarios mean change of scope and therefore need to be handled more formally (outside the SDT)
  - Changing breadth of solution
    - Adding to high level requirements
    - Removing Must Have requirements (Increment or Project Level)
  - Increasing % of Must Haves
    - By adding new Musts or upgrading Won'ts, Coulds or Shoulds to Must Have
- SDT always empowered to operate within previously agreed boundaries without need to escalate to PM



THANK YOU

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

In the Name of God for Mankind

www.utm.my

Practical Application of Agile PM Principles – Case Study

Rohayanti Hassan
Faculty of Computing
Universiti Teknologi Malaysia

This slide contains material that is copied from the book Agile Project Manager_v2_handbook by Agile Business Consortium. All rights reserved. No part of this material may be reproduced or transmitted in any form or by any means without the prior written permission of the copyright owner. This material is used here for educational purposes under the doctrine of fair use, with the intention to support learning and teaching.

For more information, please refer to the original source:
Book Title: Agile Project Manager_v2_handbook
Author: Agile Business Consortium
Publisher: Buckland Media Group Limited
Year of Publication: 2017
Please consult the book for further details and complete content.

Outline

- User Stories
- DSDM Practice : MoSCoW Prioritization
  - Introduction
  - The MoSCoW Rules
  - MoSCow relating to a Specific Timeframe

- DSDM Practice : Timeboxing
  - Timeboxing Options
  - The Daily Stand Up (Scrum)
  - Timeboxes - The Wider Context

DSDM Practice : MoSCoW Prioritisation

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

innovative • entrepreneurial • global

The Case Study

Case Study: Attendance Management System Using Facial Recognition (AMS-FR)

The Faculty of Computing plans to digitalize attendance monitoring for lectures, tutorials, and labs by implementing an **Attendance Management System using Facial Recognition (AMS-FR)**. The system replaces manual sign-in sheets with automated face detection and recognition at classroom entrances or via in-class capture devices.

When students enter the classroom, a camera captures real-time facial images and matches them against the Faculty's student database. Recognized students are instantly marked as *present*, while mismatches or unverified faces are logged for lecturer review. The system also supports late arrival tagging, absentee alerts, and generates analytics such as attendance trends, weekly attendance summaries, low-attendance warnings, and course-level dashboards.

The system supports:

- **Students** → seamless attendance capture
- **Lecturers** → automated registers, review mismatches, generate evidence
- **Administrators** → analytics, reporting, system configuration

User Story #1

Automated Facial Recognition Attendance Capture

As a Lecturer
I want the system to automatically record student attendance using facial recognition at classroom entry
So that attendance is captured accurately and without manual effort.

Functional Requirements

System must capture real-time images from the classroom camera.
System must match detected faces with registered student profiles.
System must auto-tag attendance as “Present”, “Late”, or “Unverified”.
System must allow the lecturer to review and resolve unverified attendance.
System must store attendance records in the database.

Non-Functional Requirements

Recognition accuracy ≥ 90% in normal classroom lighting.
Response time ≤ 3 seconds per face match.
System uptime ≥ 99% during lecture hours.
Data must comply with university privacy and security policies (encrypted storage).
Camera feed must handle at least 20 simultaneous face detections.

Acceptance Criteria

Given a student walks into class, when the system detects their face, then the system must match it to a valid student profile.
When a match succeeds, the system must mark the student as *Present* with a timestamp.
When a match fails, the system must mark the entry as *Unverified* and place it into the lecturer’s review panel.
The attendance status must be stored in the database and appear correctly on the lecturer dashboard.

User Story #1

Automated Facial Recognition Attendance Capture

Tasks & Subtasks

1. UI / UX Design

Design attendance dashboard for lecturers
Design unverified-attendance review screen
Design live camera feed monitoring UI

2. Backend Logic

Implement facial detection and recognition API
Implement attendance tagging logic
Build database schema for attendance records
Develop mismatch review service

3. Integration

Connect camera hardware feed to backend
Integrate facial recognition model with API
Implement authentication for lecturers

4. Testing

Unit testing facial recognition module
System testing for real-time capture
Performance test under multiple faces
Security testing on data privacy

5. Deployment

Deploy backend to server environment
Configure classroom devices (camera + client app)
Monitor logs and optimize model performance

User Story #2

Attendance Analytics Dashboard

As an Administrator
I want the system to generate attendance analytics and visual reports
So that I can monitor trends, detect low attendance patterns, and support academic decisions.

Functional Requirements

System must compute weekly and monthly attendance summaries.
System must display analytics (charts, heatmaps, trend lines).
System must allow export of analytics (PDF/CSV).
System must generate low-attendance alerts.
System must support filtering by class, lecturer, semester, and program.

Non-Functional Requirements

Dashboard must load analytics within 5 seconds.
Charts must remain readable on desktop and mobile layouts.
Reports must support export formats ≤ 5MB per file.
System must maintain data integrity with accurate calculations.
High availability during peak reporting periods.

Acceptance Criteria

The dashboard must show accurate attendance percentages for all selected classes.
When an administrator filters by semester/program, the data must update correctly.
When attendance falls below the configurable threshold, the system must generate an alert.
Exported analytics must match what is shown on screen.

User Story #2

Attendance Analytics Dashboard

Tasks & Subtasks

1. UI / UX Design

Design analytics dashboard layout
Design charts (bar, line, pie, heatmap)
Design export report interface

2. Backend Logic

Build analytics computation engine
Implement filtering and dataset aggregation
Implement alert generation service

3. Integration

Connect dashboard frontend with analytics API
Integrate report export module
Sync attendance database to ensure consistent data

4. Testing

Validate analytics calculations
Test filters and chart rendering
Test export functionality
Load testing for large datasets

5. Deployment

Deploy analytics microservice
Configure cron jobs for data aggregation
Set up monitoring for performance and errors

DSDM Practice - MoSCoW Prioritisation

The project team must analyse functional and non-functional requirements, prioritise them using MoSCoW classification.

Must have = Minimum Usable SubseT (Guaranteed)
Without this requirement . no point on this date or not legal or unsafe or not a viable solution

Should have
Important but not vital
May be painful to leave out but solution still viable
May need workaround, which may be temporary

Could have
Wanted or desirable but less important
Less impact if left out (compared with a Should)

Won't have this time
Will not be delivered in this timeframe



Functional Requirements (FR) – MoSCoW Prioritisation

M — Must Have (Essential for system operation)

These requirements are critical for the system to function. Without them, the system fails its core purpose.

1. **FR1**: Capture real-time images from the classroom camera for facial detection.
2. **FR2**: Match detected faces with registered student profiles using facial recognition.
3. **FR3**: Automatically mark student attendance as *Present*, *Late*, or *Unverified*.
4. **FR4**: Store attendance records securely in the database.
5. **FR5**: Provide lecturers with a dashboard to view real-time attendance.

S — Should Have (Important but not immediately mandatory)

These add significant value but are not critical for initial deployment.

6. **FR6**: Allow lecturers to review and resolve unverified or mismatched attendance.
7. **FR7**: Generate attendance analytics (weekly/monthly summaries).
8. **FR8**: Support filtering (course, section, semester).
9. **FR9**: Allow export of attendance reports (CSV/PDF).

C — Could Have (Desirable but lower importance)

Enhancements that improve user experience but are not essential.

10. **FR10**: Provide mobile-responsive dashboards.
11. **FR11**: Integrate with student mobile app for notifications (absent alerts).
12. **FR12**: Provide real-time heatmap visualisation of attendance trends.

W — Won't Have (Not planned for current release)

Listed to prevent scope creep.

13. **FR13**: Integration with biometric devices other than facial recognition (e.g., fingerprints).
14. **FR14**: Inter-faculty integration for a university-wide attendance system (future phase).

Non-functional Requirements (FR) – MoSCoW Prioritisation

M — Must Have

1. **NFR1 (Performance)**: Facial recognition response time ≤ 3 seconds per person.
2. **NFR2 (Accuracy)**: Facial recognition accuracy ≥ 90%.
3. **NFR3 (Security)**: All personal data must be encrypted (at rest + in transit).
4. **NFR4 (Availability)**: System uptime ≥ 99% during lecture periods.

S — Should Have

5. **NFR5 (Scalability)**: System should support at least 20–30 face detections simultaneously.
6. **NFR6 (Usability)**: Interface should follow faculty UI standards and be lecturer-friendly.
7. **NFR7 (Compatibility)**: System should run on standard classroom hardware (CCTV/IP camera + laptop).

C — Could Have

8. **NFR8 (Portability)**: Admin dashboard could support tablets for mobile monitoring.
9. **NFR9 (Maintainability)**: System could auto-generate system health logs for maintenance.

W — Won't Have

10. **NFR10**: Cloud auto-scaling architecture for large multi-campus operations (future enhancement).

Exercise #1: Consolidated (FR + NFR) MoSCoW Table

Req ID	Requirements	Moscow Classification	Justification

Requirement	Description	MoSCoW Priority
FR1	Real-time image capture	Must
FR2	Facial recognition matching	Must

*** User Microsoft Excel**

Exercise #2: Add on with Tasks and Subtasks

Req ID	Requirements	Moscow Classification	Task	Sub Task

Req ID	Requirement Description	Priority (MoSCoW)	Tasks	Sub-Tasks
FR1	Real-time image capture from camera	Must	Develop camera integration module	• Connect IP/CCTV camera feed • Implement continuous frame capture • Error handling for camera offline

*** User Microsoft Excel**

DSDM Practice : Timeboxing

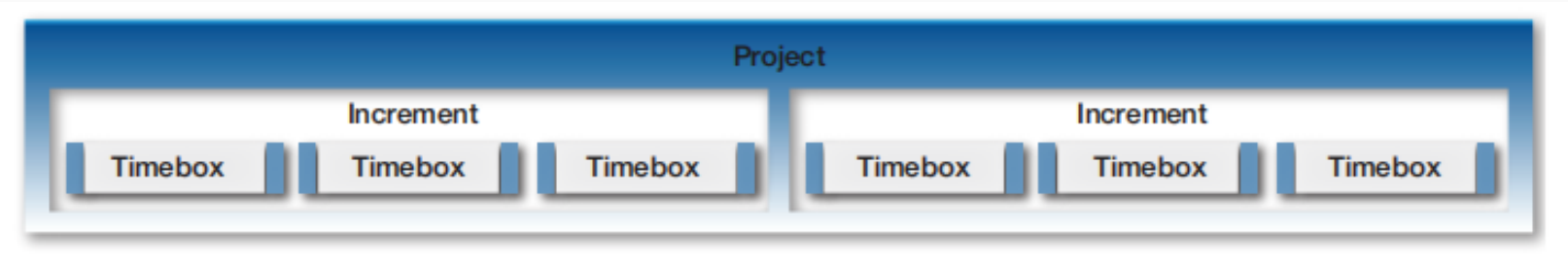
UTM
UNIVERSITI TEKNOLOGI MALAYSIA

innovative • entrepreneurial • global

Timeboxes - How Long?

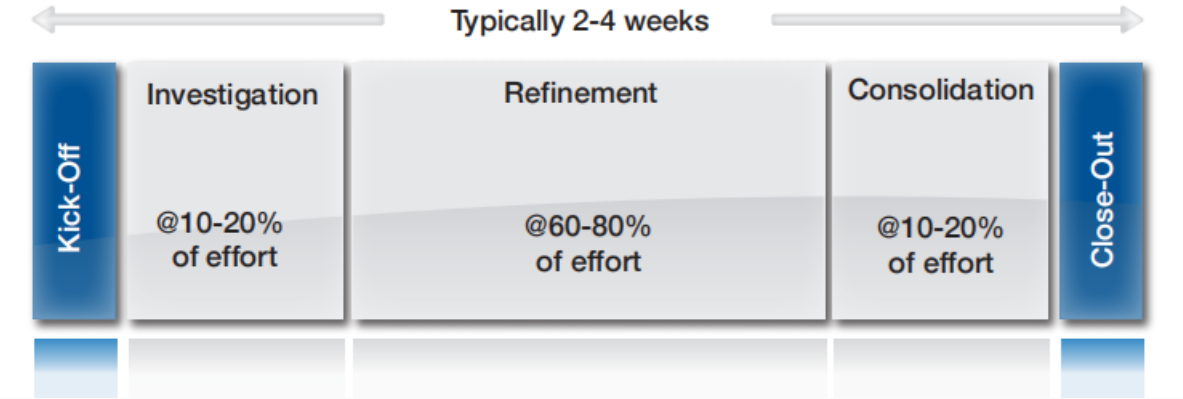
During Foundations, Agile PM helps Solution Development Team agree preferred length of Timebox

- Short enough to keep team focused
- Long enough to achieve something complete and meaningful
- Usually between 2 and 4 weeks
- Longer Timeboxes should be avoided unless really necessary



Timeboxing - Which style?

- DSDM Structured Timebox



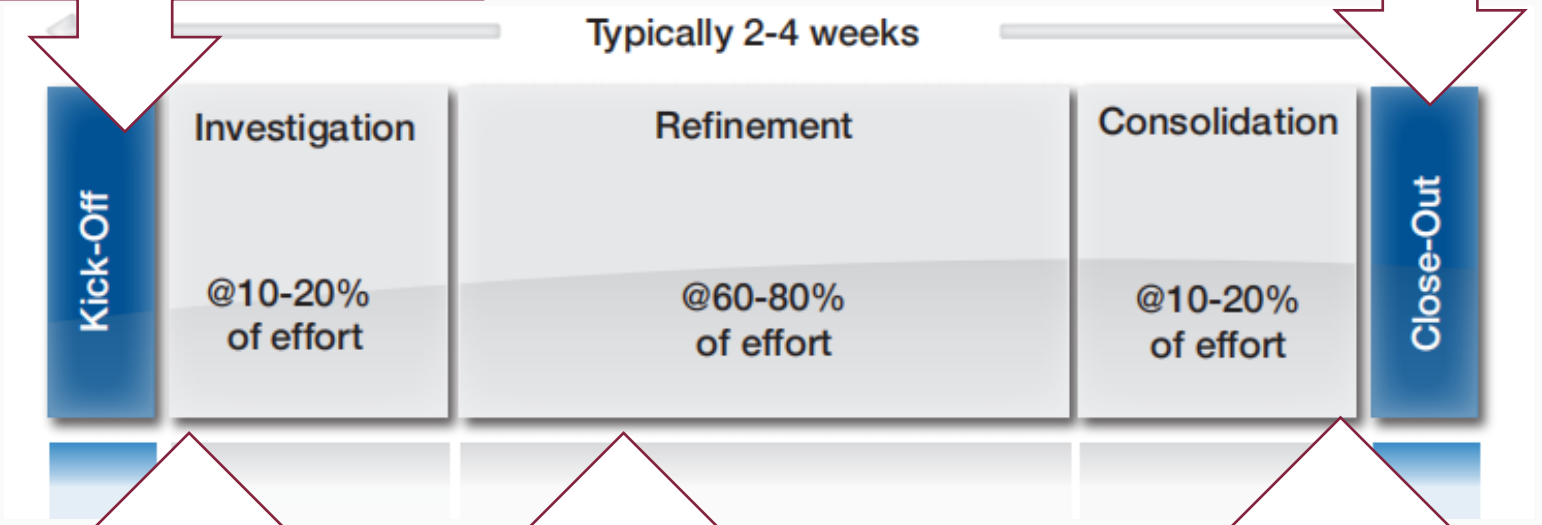
- Free Format Timebox



A DSDM Structured Timebox

- Check TB objectives in Delivery Plan still feasible
- Agree acceptance criteria, where possible
- Review availability of SDT members
- Highlight known dependencies

- Sign-off what has been delivered
- Assess impact of what has not been "done"
- Build on good experience
- Avoid repeating same mistakes
- Define issues to be resolved for next TB



- Understand detail and priorities of work to be done
- Review dependencies, Timebox Plan and risks

- Work on the Solution in line with agreed MoSCoW priorities
- Review at end of Refinement (work should be nearly ready)

- Complete actions from Refinement review
- Complete final testing
- Ensure organisational or project standards have been met, with any final quality checks
- Review at end of Consolidation, ensuring TB objectives are met.
- Any products not meeting acceptance criteria deemed as "Not Delivered"

Timebox Plan

There are two Solution Development Teams, and each team is assigned one user story. The project duration is one month, and each sprint lasts two weeks. Please prepare a detailed timebox plan for both Solution Development Teams.

⭐ Overview

Team	User Story	Focus Area
Team A	User Story 1	Automated Facial Recognition Attendance Capture
Team B	User Story 2	Attendance Analytics Dashboard

Project Timeline: **1 Month = 4 Weeks = 2 Sprints**

🟦 TIMEBOX PLAN – SPRINT 1 (Weeks 1–2)

🎯 Sprint 1 Objective

- Establish core system foundations for each user story.
- Deliver initial working components for recognition and analytics.

TB 1: TEAM A – User Story 1: Automated Attendance Capture

INVESTIGATE (Days 1–3)

- Analyse classroom camera requirements
- Validate facial recognition model feasibility
- Identify attendance states (Present, Late, Unverified)
- Draft initial database schema for attendance records

REFINE (Days 4–10)

- Integrate camera feed (live capture)
- Develop facial detection & matching prototype
- Implement attendance tagging logic (version 1)
- Create basic lecturer dashboard for viewing attendance

CONSOLIDATE (Days 11–14)

- Debug issues in face detection
- Improve recognition threshold accuracy
- Conduct initial classroom testing
- Deploy Sprint 1 build to staging & conduct sprint review

Sprint 1 Deliverables (Team A)

- ✔ Camera integration
- ✔ Basic facial detection engine
- ✔ Attendance tagging v1
- ✔ Basic dashboard view

TB 1: TEAM B – User Story 2: Attendance Analytics Dashboard

INVESTIGATE (Days 1–3)

- Define analytics requirements: weekly %, monthly %, trend data
- Draft dashboard wireframes
- Confirm data structure and filtering fields (course/section/semester)

REFINE (Days 4–10)

- Develop analytics computation engine (version 1)
- Build chart prototypes (bar chart, line chart)
- Implement filtering backend (course/section/semester)
- Create initial dashboard layout

CONSOLIDATE (Days 11–14)

- Validate accuracy of analytics data
- Fix UI layout and chart rendering issues
- Improve dashboard responsiveness
- Deploy Sprint 1 build to staging & conduct sprint review

Sprint 1 Deliverables (Team B)

- ✔ Analytics engine v1
- ✔ Filters implemented
- ✔ Chart prototypes
- ✔ Basic dashboard shell

TB 2: TEAM A – User Story 1: Automated Attendance Capture

INVESTIGATE (Days 1–3)

- Review Sprint 1 issues: accuracy gaps, performance delays
- Analyse multi-face detection scenarios
- Identify security requirements (encryption, HTTPS)

REFINE (Days 4–10)

- Develop "Unverified Attendance Review Panel"
- Implement manual verification & correction options
- Improve performance to ≤ 3 seconds per recognition
- Enhance dashboard with real-time updates
- Integrate secure API connections

CONSOLIDATE (Days 11–14)

- Conduct load testing (20–30 faces simultaneously)
- Achieve ≥ 90% facial recognition accuracy
- Final system integration testing
- Sprint review & production deployment

Sprint 2 Deliverables (Team A)

- ✔ Full facial recognition workflow
- ✔ Unverified attendance review panel
- ✔ Optimised dashboard
- ✔ Performance & accuracy improvements

TB: TEAM B – User Story 2: Attendance Analytics Dashboard

INVESTIGATE (Days 1–3)

- Finalize export formats (PDF, CSV)
- Define low attendance alert rules (thresholds)
- Identify heatmap visual design

REFINE (Days 4–10)

- Implement export module (PDF/CSV)
- Build weekly/monthly summary report generator
- Develop low attendance alert module
- Create attendance heatmap visualization
- Improve UX for dashboard navigation

CONSOLIDATE (Days 11–14)

- Correct export formatting errors
- Validate consistency of analytics and reports
- Optimize UI responsiveness (desktop/tablet/mobile)
- Sprint review & deploy final version to production

Sprint 2 Deliverables (Team B)

- ✔ Full analytics dashboard
- ✔ Export (PDF/CSV) working
- ✔ Low-attendance alert engine
- ✔ Heatmap analytics
- ✔ Fully responsive UI

Exercise #3: Generate Gantt chart for Timebox Plan

Using Microsoft Excel, generate a Gantt chart for the timebox plan for each team.

Include all activities and the key milestones to be delivered.

Exercise #4: Project Extension

✅ 1. New Additional User Story (User Story 3)

User Story 3 – Student Self-Service Attendance & Notification Portal

As a Student

I want to view my personal attendance history and receive automated notifications

So that I can monitor my attendance status and take corrective actions when necessary.

Functional Requirements

- FR15: Students can log in to view daily, weekly, and semester attendance.
- FR16: System sends automated notifications for low attendance warnings.
- FR17: Students can download their attendance summary (PDF).
- FR18: Students can request correction if wrongly marked as absent (optional future enhancement).

Non-Functional Requirements

- NFR11: Dashboard must load within 5 seconds.
- NFR12: Notification delivery rate ≥ 98%.
- NFR13: Student data must remain private and secured by access control (role-based).
- NFR14: Interface must be mobile-responsive for smartphones.

Acceptance Criteria

- Students can view all attendance records for the semester with accurate percentages.
- When attendance drops below the configured threshold, the system triggers a notification.
- PDF downloads must match attendance displayed in the system.
- Only authenticated students may access their own data.

The project is extended for one additional Sprint (Sprint 3) to incorporate a new user story involving a Student Self-Service Attendance & Notification Portal. This extension aims to enable students to view their attendance, receive automated low-attendance notifications, and download attendance summaries. Please prepare an **additional timebox plan for Sprint 3** to accommodate the new user story and deliver the required features within a two-week sprint.



THANK YOU

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

In the Name of God for Mankind

www.utm.my

DSDM Practise :

- Facilitated Workshops
- Modelling
- Iterative Development

Rohayanti Hassan
Faculty of Computing
Universiti Teknologi Malaysia

This slide contains material that is copied from the book Agile Project Manager_v2_handbook by Agile Business Consortium. All rights reserved. No part of this material may be reproduced or transmitted in any form or by any means without the prior written permission of the copyright owner. This material is used here for educational purposes under the doctrine of fair use, with the intention to support learning and teaching.

For more information, please refer to the original source:
Book Title: Agile Project Manager_v2_handbook
Author: Agile Business Consortium
Publisher: Buckland Media Group Limited
Year of Publication: 2017
Please consult the book for further details and complete content.

Outline

- Other DSDM Practices :
  - Facilitated Workshops
  - Modelling
  - Iterative Development

Facilitated Workshops

*"A structured group meeting to enable participants to reach a predetermined objective in a compressed time frame, supported by a **neutral facilitator**."*

Success Factors

- Effective, trained, independent Workshop Facilitator
- Flexibility in format, within clearly defined objectives
- Thorough preparation (by Facilitator and Participants)
- Ability to build in outputs from previous workshops
- Decisions and agreements not forced
- Workshop report distributed to participants soon afterwards



Facilitated Workshops

*"A structured group meeting to enable participants to reach a predetermined objective in a compressed time frame, supported by a **neutral facilitator**."*

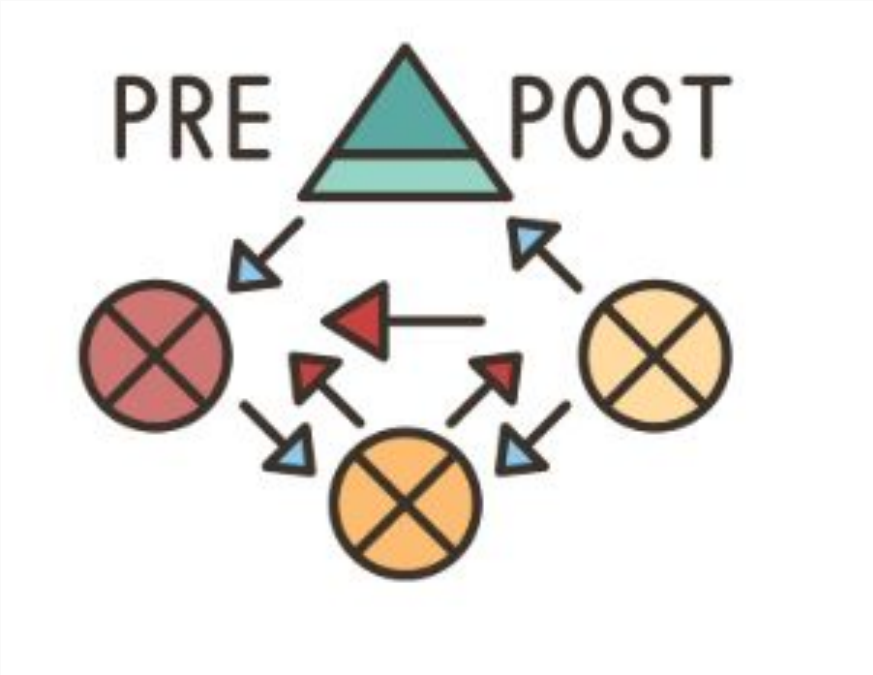


Benefits

- Rapid, high quality team decisions
- Greater buy-in and ownership from all stakeholders
- Clarification of issues and enhanced communication
- Building team spirit and consensus

Key DSDM Practice: Modelling

- Visual representation of problem or solution
- Description or analogy to help visualise something that cannot be directly observed.
- A small but exact copy of something
- A pattern or figure of something to be made
- Often used to describe a set of diagrams



Key DSDM Practice: Modelling

Purpose of Modelling

- Improves communication and prompts the right questions.
- Helps confirm expectations, test achievability of objectives, and validate requirements.
- Makes solution elements visible early (e.g., diagrams, prototypes, mock-ups).
- Should only be detailed enough to serve the purpose—avoid unnecessary effort.
- Often involves **abstraction**: omitting details to focus on specific aspects.



Modelling Perspectives

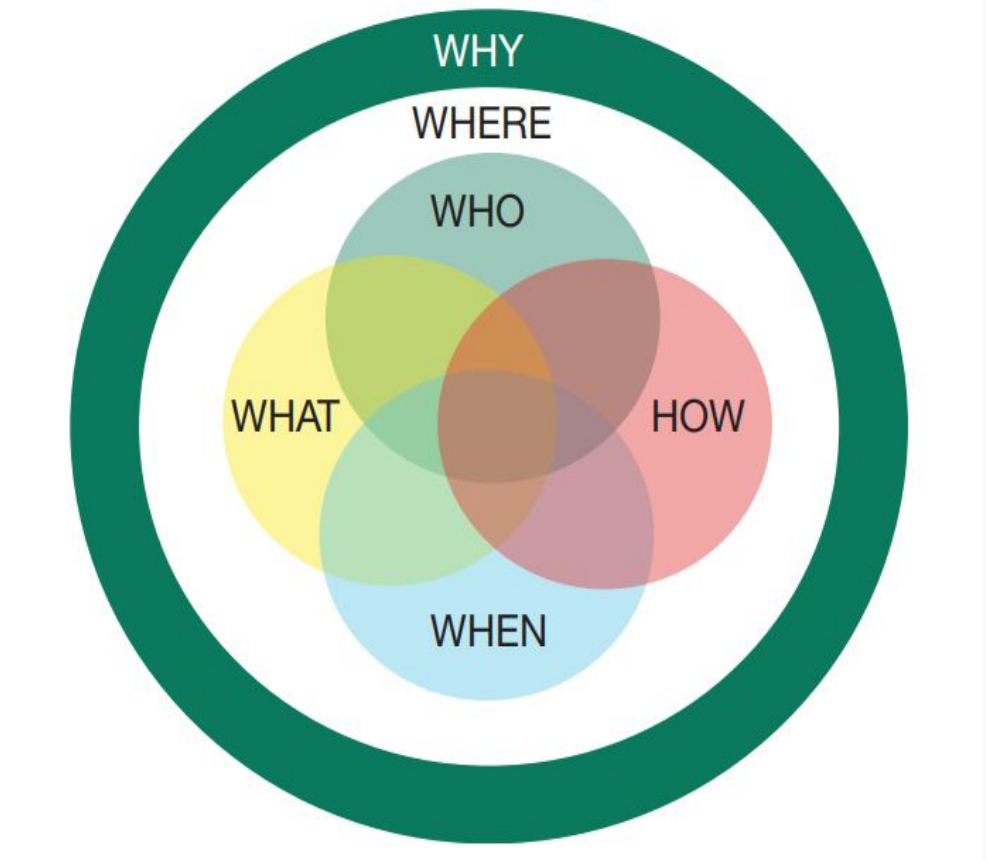
- Provides a **coherent picture of a solution area**.
- Considers perspectives: **What, How, Where, Who, When, Why**.
- Shows relationships between these perspectives.

Key DSDM Practice: Modelling

Diagram: Modelling Perspectives

A Venn diagram shows the six perspectives:

- **WHAT** – Information within the solution area (data, relationships, business rules).
- **HOW** – Functions, features, and processes.
- **WHERE** – Locations of business operations (in relation to the solution).
- **WHO** – People (customers, users, stakeholders).
- **WHEN** – Events important to the business (time, scheduling).
- **WHY** – Business objectives and strategy.



Key DSDM Practice: Iterative Development

- Solution evolves through cycles of iterative development
- Each cycle moves solution from high-level concept closer to something with acknowledged business value
- Each cycle should be:
  - As short as possible
  - Only as formal as necessary
- Typically after an initial conversation, follow cycles of

Thought, Action and Conversation

Key DSDM Practice: Iterative Development

Planning Iterative Development

- Strategy decided during **Foundations phase**
- Large solution broken into **small manageable chunks**
- Work delivered in **Timeboxes** (short, fixed periods)
- Each Timebox contains prioritized requirements/user stories
- **Benefits**:
  - Easier progress tracking
  - Faster delivery of usable increments
  - Reduced project risks

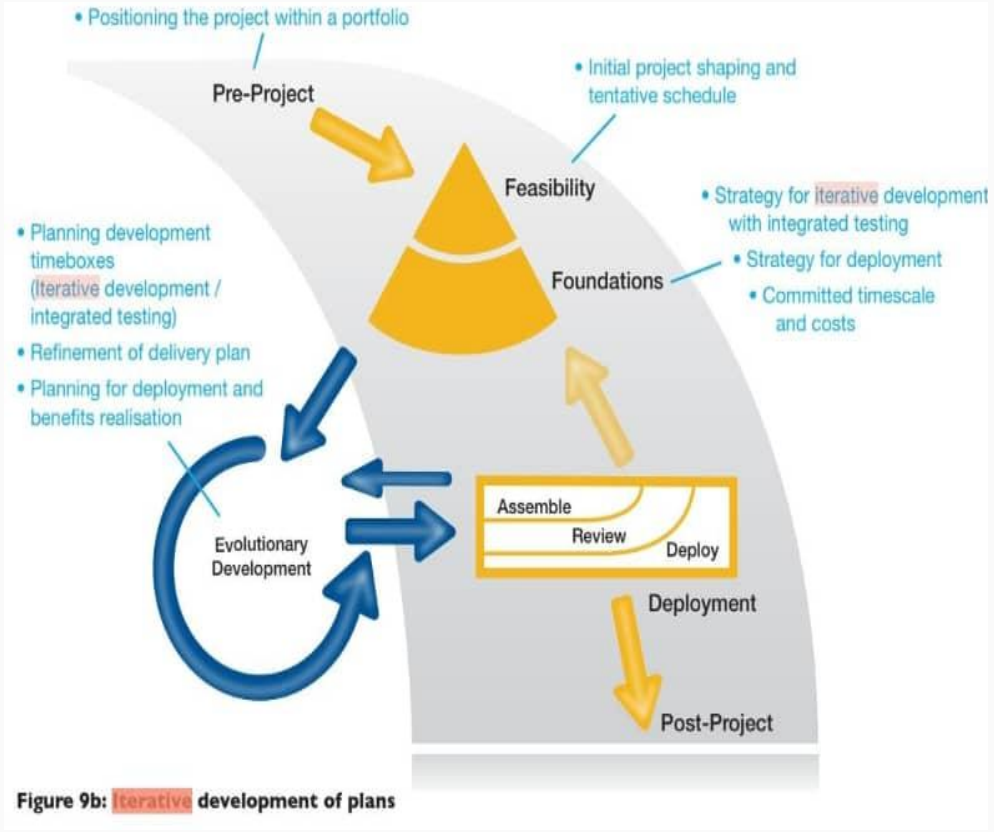


Figure 9b: Iterative development of plans

Key DSDM Practice: Iterative Development

Controlling Iterative Development



- Each cycle delivers **incremental improvements**
- Reviews may show:
  - Wrong evolution of solution
  - Changing business needs
  - New issues discovered
- **Configuration Management** ensures correct versions and stability
- **Benefits**:
  - Maintains solution integrity
  - Encourages flexibility and adaptability

Key DSDM Practice: Iterative Development



Quality in Iterative Development

- Core rule: **Never compromise quality**
- Quality defined **early** and verified **continuously**
- **Verification methods**:
  - **Reviews (static):** check documents, designs, code
  - **Testing (dynamic):** run deliverables to confirm behavior
- **Benefits**:
  - Detects defects early
  - Builds stakeholder confidence
  - Ensures solution is **fit for purpose**

Key DSDM Practice: Iterative Development

Reviews & Testing

- **Reviews**: from informal peer checks → structured formal reviews
- **Types of Tests**:
  - **Positive** – confirm expected behavior
  - **Negative** – block invalid actions
  - **Unhappy Path** – test unusual cases
- **Good tests include**:
  - Starting state
  - Set of actions
  - Expected outcome
- **Benefits**:
  - Improves reliability
  - Reduces risk of failure



Key DSDM Practice: Iterative Development

Roles & Responsibilities

- **Technical Coordinator**
  - Ensures quality from technical perspective
  - Validates non-functional requirements are met
  - Oversees technical reviews and rigorous testing

- **Solution Tester**
  - Responsible for most testing activities
  - Exceptions:
    - Business acceptance testing → Business Ambassadors/Advisors
    - Unit testing → Solution Developers
  - Provides testing knowledge & expertise to the team

Key DSDM Practice: Iterative Development

Iterative Development & Incremental Delivery

- Each Timebox should ideally deliver a **potentially deployable increment**
- Testing integrated into development & delivery process
- Benefits:
  - Reduces project risks
  - Supports early **return on investment** through incremental delivery



Key DSDM Practice: Iterative Development

Investigate, Refine, Consolidate (IRC)

- Used for each requirement/user story/product
- **Investigate** – understand details, define acceptance criteria
- **Refine** – develop and enhance solution
- **Consolidate** – converge on accurate solution
- Helps avoid excessive iterations
- Reviews scheduled within Timebox → maintain business focus & stakeholder engagement

Questions on other practises?



EXERCISES

1. What is Iterative Development?

   a) A practice where features are designed, developed and tested
   b) A gated process where development can not begin until the finished design is complete
   c) An early sample, model or release of a product to test a concept or process
   d) The process and activity of planning, organizing, motivating and controlling resources to achieve a specific goal

1. In a free format Timebox, which activity is carried out during the Iterative Development step?

   a) Agree what work will be undertaken, agree priorities and accept the workload
   b) Scheduled reviews of the work in progress
   c) Formal acceptance of the Timebox deliverables
   d) Workshop to learn from experience and to take action to improve future Timeboxes

EXERCISES

3. During Iterative Development, which testing activity is the responsibility of the Solution Tester?

   a) Business acceptance testing
   b) Unit testing of a feature
   c) Ensuring that testing is sufficiently comprehensive
   d) Providing testing knowledge and expertise

4. When using Iterative Development, what should each cycle begin and end with?

   a) Complete solution
   b) Conversation
   c) Facilitated Workshop
   d) Timebox

EXERCISES

5. Identify the missing word(s) in the following sentence:

   A DSDM project will always deliver a viable solution, as long as the practices of [?] and Timeboxing are followed.

   a) Daily Stand-ups
   b) MoSCoW
   c) Iterative Development
   d) Incremental Delivery



THANK YOU

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

In the Name of God for Mankind

www.utm.my

Software Project Estimation

Rohayanti Hassan
Faculty of Computing
Universiti Teknologi Malaysia

This slide contains material that is copied from the book Agile Project Manager_v2_handbook by Agile Business Consortium. All rights reserved. No part of this material may be reproduced or transmitted in any form or by any means without the prior written permission of the copyright owner. This material is used here for educational purposes under the doctrine of fair use, with the intention to support learning and teaching.

For more information, please refer to the original source:
Book Title: Agile Project Manager_v2_handbook
Author: Agile Business Consortium
Publisher: Buckland Media Group Limited
Year of Publication: 2017
Please consult the book for further details and complete content.

Outline

- Understanding Software Effort Estimation Basics
- Factors Affecting Software Development Costs
- Proven Techniques for Estimating Project Budgets
- Top-Down Estimation
- Bottom-Up Estimation

Learning Outcomes

By the end of this chapter, you will be able to:

- Understand the fundamentals of software effort and cost estimation
- Explain key estimation concepts such as effort, cost, size, and complexity
- Compare traditional estimation models with agile-based estimation approaches
- Apply estimation techniques such as T-Shirt,Velocity,Function Point and others.
- Recognize factors that influence estimation accuracy
- Appreciate the importance of estimation in planning, budgeting, and monitoring software projects

Understanding Software Effort Estimation Basics



What is Software Effort Estimation?

Software Effort Estimation refers to predicting the amount of work, time, and resources required to develop a software product.

- Helps determine project feasibility
- Supports budgeting and scheduling
- Essential for planning releases in both traditional and agile projects
- Mentioned as a critical early-stage activity in research (Rivera Ibarra et al., 2024)



Purpose of Software Effort Estimation



Estimation provides a foundation for making informed decisions about project planning and delivery. It reduces uncertainty and helps teams prepare for the work ahead.

- Plan project timelines
- Allocate team members and resources
- Estimate cost and effort
- Reduce project risks and uncertainties
- Support decision-making for contracts and budgeting

(Reference: Mike Cohn, Agile Estimating and Planning)

Importance of Software Estimation



Accurate estimation improves planning and ensures projects deliver value within realistic constraints. It helps both teams and stakeholders set expectations.

- Prevents overruns in cost and schedule
- Enables better resource planning
- Helps manage stakeholder expectations
- Supports delivery of high-value features

Challenges in Estimation

Software development involves uncertainty, changing requirements, and human judgement —all of which make estimation difficult.



- Requirements may be unclear early
- Technical complexity is hard to predict
- Human bias affects estimates
- Teams may lack historical data
- Agile projects constantly evolve

Effort vs Duration vs Cost

Although related, effort, duration, and cost measure different aspects of a project and should not be confused.

- Effort: Human labor required
- Duration: Calendar time including delays
- Cost: Monetary value based on effort and resources
- Effort does not always equal duration



Factors Affecting Software Development Costs



What Effects Software Development Costs?

Software development costs are influenced by many internal and external factors. These factors impact effort, timeline, resources, and estimation accuracy across projects.

- Product characteristics
- Project environment
- Technical complexity
- Human and team factors
- Organizational maturity
- Lessons from empirical studies



Product Factors



Product-related attributes significantly influence how much effort a project requires. Larger or more complex products naturally demand more time and resources.

- System size (SLOC, FP, Story Points)
- Functional complexity (features, interfaces)
- Required performance, reliability, and security
- Number of modules or integrations

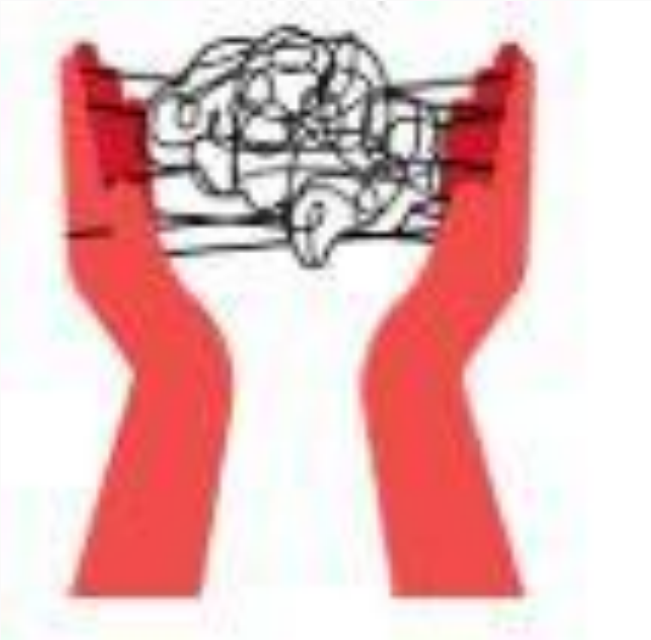
Project Environment Factors

The environment surrounding the project also affects cost. These conditions shape how smoothly development can proceed.

- Tools, frameworks, and technologies used
- Agile vs traditional environment difference
- Availability of domain experts
- Requirements clarity or volatility
- Contract type (flexible vs fixed) impacts estimation variation



Technical Complexity Modeling



Technical complexity is a major cost driver because complex software demands more effort in design, coding, testing, and integration.

- Complex algorithms or data structures
- Multiple system interfaces
- Security and compliance needs
- Integration with legacy systems
- Hidden complexity increases effort (Scrum SLR finding)

Human & Team Factors

People-related factors play a major role. Skills, communication, and team structure have direct effects on cost and productivity.

- Team experience influences accuracy and productivity (ranked highest factor)
- Skill level with programming tools and languages
- Communication overhead (Brooks's Law effect)
- Coordination challenges in larger teams
- Team cohesion impacts COCOMO scale factors (TEAM)



Time & Scheduling Factors

How time is allocated affects costs—both in agile and traditional environments.

- Shorter, iterative cycles reduce large uncertainty
- Delays in one stage influence downstream tasks
- Verification & validation effort distribution differs (Agile vs Traditional)
- Changing deadlines add cost pressures



Organizational Maturity



Organizations with well-established processes experience lower cost variation.

- Use of standards (CMMI, structured processes)
- Experience handling Agile in acquisition environments
- Better risk identification and management
- Clear communication between teams and stakeholders

Relevant insight from Agile acquisition study: Organizations lacking maturity struggle with estimation consistency.

Summary: Combined Cost Influencing Factors

Software costs are shaped by many dimensions that interact across the product, team, and organization.

- Larger and more complex products cost more
- Experienced teams estimate and deliver more accurately
- Organizational maturity enhances predictability
- Technical complexity drives testing and integration cost
- Agile processes reduce uncertainty but require team alignment
- Research shows experience, complexity, time, and size as core drivers

All factors must be considered for realistic software cost estimation.

Proven Techniques for Estimating Project Budgets



What Are Budget Estimation Techniques?

Estimation techniques help teams predict cost and effort more accurately before development begins. They provide structure so teams don't rely on guesswork. Different techniques suit different levels of detail and project types.

- Improve prediction of effort and cost
- Reduce uncertainty during planning
- Support informed decision-making



Why These Techniques Matter



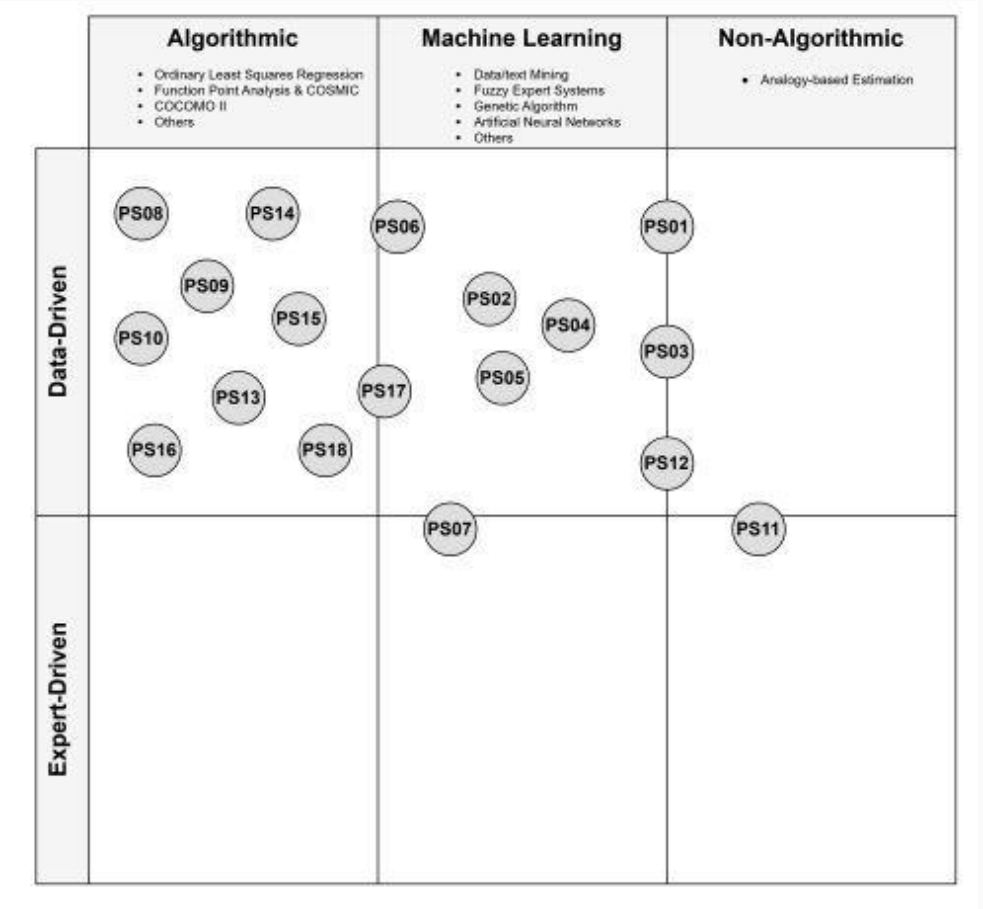
Using proven estimation methods helps teams avoid underestimation and project overruns. Teams gain clearer understanding of workload and risks. These techniques also improve planning confidence for stakeholders.

- Avoid unrealistic commitments
- Identify risk early
- Improve communication with stakeholders

Categories of Estimation Techniques

Agile estimation methods fall into several major categories. Each category offers a different way to measure effort or size. Understanding them helps teams choose the right method for budgeting and planning.

- Story-based / relative estimation
- Time-based estimation
- Probabilistic / flow forecasting
- Expert-judgment methods
- Hybrid / model-assisted approaches



Top-Down Estimation



What Is Top-Down Estimation?

Top-down estimation starts with a big-picture view of the project and gives a quick overall effort estimate. It is used early when details are not fully known.

- Estimate from overall system
- Fast and simple
- Used early in planning
- Uses T-shirt sizing & high-level Story Points



Types of Top-Down Estimation



Top-down estimation uses visible features of the product to estimate effort based on size groups and similarity.

- T-Shirt Estimating (group items into S, M, L, XL)
- Function Point Estimating
- Velocity-Based Estimation

1. T-Shirt Estimation

What Is T-Shirt Sizing?

T-shirt sizing assigns categories like S, M, L, and XL to represent effort levels. It is a coarse estimation method used when details are not clear. Teams often use it for epics or early-stage planning.

- Uses S–XL categories
- Very simple and fast
- Ideal for high-level estimation



1. T-Shirt Estimation



Why Teams Use T-Shirt Sizing

Teams use T-shirt sizing because it avoids over-analysis and quickly shows which items are large or small. It helps stakeholders understand scope without technical detail. Later, teams convert sizes into Story Points.

- Quick scope comparison
- Good for discussions with stakeholders
- Helps identify large epics early

1. T-Shirt Estimation - Example

Concept

- Requirements/stories are grouped into size categories (S, M, L, XL).
- Each size is assigned an estimated effort (e.g., in 6-hour days).
- Total effort = Size Effort × Quantity of Stories.

AMS-FR Case Study: Sample T-Shirt Estimation

User Stories

1. **FR1**: Capture student facial image during check-in → *Small*
2. **FR2**: Validate student identity using facial recognition model → *Medium*
3. **FR3**: Auto-generate attendance status and sync to dashboard → *Medium*

Assigned Effort

- Small = **1 day**
- Medium = **3 days**

Effort Calculation

Story	Size	Effort
FR1	S	1 × 1 = **1 day**
FR2	M	1 × 3 = **3 days**
FR3	M	1 × 3 = **3 days**

Total Estimated Effort = 1 + 3 + 3 = 7 days

2. Velocity-Based Estimation

What Is Velocity-Based Release Planning?

Velocity measures how many Story Points a team completes each sprint. Release planning uses this velocity to forecast how long it will take to finish the product backlog. It becomes more accurate after several sprints.

- Based on completed Story Points
- Used to predict release timelines
- Improves as team consistency grows



2. Velocity-Based Estimation



Why Teams Use Velocity

Teams use velocity because it provides data-driven forecasting instead of guesswork. It allows predictable budgeting and helps identify when a scope change affects delivery. Velocity also supports continuous improvement.

- Helps forecast future work
- Tracks changes in team performance
- Supports realistic planning conversations

2. Velocity-Based Estimation - Example

Example (Given in Text)

If a team delivers **20 story points per Timebox**, and the next Increment has **5 Timeboxes**, expected output:
20 × 5 = 100 story points

AMS-FR Case Study: Sample Velocity Calculation

User Stories

Story	Description	Story Points
FR1	Capture facial image	3
FR2	Facial recognition matching	8
FR3	Auto-generate attendance status	5
FR4	Sync to dashboard + reporting	5

Team's Past Velocity (Observed Over 3 Timeboxes)

- TB1: 18 pts
- TB2: 22 pts
- TB3: 20 pts

Average Velocity = (18 + 22 + 20) ÷ 3 = 20 story points per Timebox

---

Concept

- ❖ Velocity = Average number of story points a team delivers per Timebox.
- ❖ Used to forecast how much work can be completed in upcoming Timeboxes

- ❑ Velocity is unique to each team and cannot be compared across teams.
- ❑ Changes in team composition require recalculating velocity.
- ❑ For new teams, velocity is estimated during Timebox Planning by sizing and filling the Timebox until capacity is reached.

Team's Past Velocity (Observed Over 3 Timeboxes)

- TB1: 18 pts
- TB2: 22 pts
- TB3: 20 pts

Average Velocity = (18 + 22 + 20) ÷ 3 = 20 story points per Timebox

---

Forecast for Next Timebox

Total AMS-FR story points = **3 + 8 + 5 + 5 = 21 points**

With a velocity of **20 points**, the forecast is:

- ✔ **The team can deliver ~20 points in the next Timebox**
- ✔ Remaining **1 point** moves to the next Timebox or is absorbed if capacity allows

---

3. Function Point Estimation

What Is a Function Point?

A Function Point measures the amount of functionality a system provides to the user. It counts things like inputs, outputs, and data files to show how big the system is. It does not depend on programming language.

- Measures system size
- Based on user-visible functions
- Independent of technology



3. Function Point Estimation



Why Teams Use Function Points

Teams use Function Points to estimate work early before coding begins. FP helps them understand system size and plan cost or effort. It gives a clear and consistent way to compare projects.

- Measures system size
- Based on user-visible functions
- Independent of technology

3. Function Point Estimation



Components of Function Points

FP is calculated by identifying different types of system functions. Each function type contributes a certain number of points depending on complexity. These components reflect how data and processes flow in the system.

- External Inputs (EI)
- External Outputs (EO)
- External Inquiries (EQ)
- Internal Logical Files (ILF)
- External Interface Files (EIF)

3. Function Point Estimation - Example

Case Study: AMS-FR (Attendance Management System – Facial Recognition)

Step 1: Identify System Boundary

The AMS-FR system includes:

- Student facial check-in
- Facial recognition validation
- Attendance recording
- Analytics dashboard & reports
- Integration with student database

Step 2: Identify Function Types

Using standard IFPUG components:

Function Type	Description in AMS-FR
EI (External Input)	Data entering the system
EO (External Output)	Data leaving the system with processing
EQ (External Inquiry)	Data retrieval with minimal processing
ILF (Internal Logical File)	Internal data maintained by system
EIF (External Interface File)	External data used but not maintained

Step 3: Map AMS-FR Functions to FP Components

External Inputs (EI)

No	Function	Complexity
EI1	Capture student facial image during check-in	Medium
EI2	Lecturer manually update attendance (override)	Low

Total EI = 2

External Outputs (EO)

No	Function	Complexity
EO1	Auto-generate attendance status	Medium
EO2	Generate attendance analytics report	High

Total EO = 2

3. Function Point Estimation - Example

External Inquiries (EQ)

No	Function	Complexity
EQ1	Lecturer view student attendance record	Low
EQ2	Search attendance by date/class	Low

Total EQ = 2

Internal Logical Files (ILF)

No	Data File	Complexity
ILF1	Student Profile Database	Medium
ILF2	Attendance Records	Medium
ILF3	Facial Feature Templates	High

Total ILF = 3

External Interface Files (EIF)

No	External File	Complexity
EIF1	University Student Information System	Medium

Total EIF = 1

Step 4: Apply Standard FP Weights

Component	Low	Medium	High
EI	3	4	6
EO	4	5	7
EQ	3	4	6
ILF	7	10	15
EIF	5	7	10

3. Function Point Estimation - Example

Step 5: Calculate Unadjusted Function Points (UFP)

EI Calculation

- EI1 (Medium): 4
- EI2 (Low): 3

  EI Total = 7

EO Calculation

- EO1 (Medium): 5
- EO2 (High): 7

  EO Total = 12

EQ Calculation

- EQ1 (Low): 3
- EQ2 (Low): 3

  EQ Total = 6

ILF Calculation

- ILF1 (Medium): 10
- ILF2 (Medium): 10
- ILF3 (High): 15

  ILF Total = 35

EIF Calculation

- EIF1 (Medium): 7

  EIF Total = 7

Step 6: Total Unadjusted Function Points

$$\mathbf{UFP} = 7 + 12 + 6 + 35 + 7 = \boxed{67 \text{ Function Points}}$$

(Optional – for Advanced Students)

You may later introduce:

- **Value Adjustment Factor (VAF)**
- **Adjusted FP = UFP × VAF**
- Conversion to effort:

  `Effort (days) = FP × productivity rate (e.g. 0.5 day/FP)`

Example:

$$67 \times 0.5 = 33.5 \text{ person-days}$$

Bottom-Up Estimation



What Is Bottom-Up Estimation?

Bottom-up estimation starts by breaking work into small tasks and estimating each one. It is used when the team has clear details of the work.

- Estimate from small tasks
- More accurate
- Used later when details are known
- Uses task breakdown & detailed sizing



Types of Bottom-Up Estimation



Bottom-up estimation breaks work into smaller pieces and estimates each one separately.

- Work Breakdown Estimating (estimate each task)
- Product Breakdown Estimating

1. Product Breakdown Estimating

What is Product Breakdown Estimating

Product Breakdown Estimating divides the system into smaller components or sub-products. Each component is then estimated separately. This approach is helpful when dealing with large or complex systems.



- Breaks work into smaller tasks
- Estimates each task individually
- Adds tasks to get total effort

1. Product Breakdown Estimating



Why Teams Use Product Breakdown Estimating

Teams use Product Breakdown Estimating to simplify large features into manageable pieces. It helps ensure no major part of the system is forgotten. This method improves clarity and planning accuracy for big projects.

- Simplifies complex systems
- Ensures complete coverage of components
- Improves planning for large features

2. Work Breakdown Estimating

What Is Work Breakdown Estimating?

Work Breakdown Estimating breaks a feature into small, clear tasks and estimates each task separately. It helps teams understand the detailed work involved before development begins. The total effort is calculated by adding all task estimates together.



- Breaks work into smaller tasks
- Estimates each task individually
- Adds tasks to get total effort

2. Work Breakdown Estimating



Why Teams Use Work Breakdown Estimating

Teams use Work Breakdown Estimating because it gives more accurate estimates and reduces missed work. It also helps teams see what needs to be done in detail. This method is especially useful during sprint planning.

- Produces more accurate estimation
- Helps identify all required tasks
- Useful for detailed sprint planning

2. Work Breakdown Estimating - Example

Case Study: AMS-FR (Attendance Management System with Facial Recognition)

Step 1: Identify Major Product Components

The AMS-FR system is decomposed into major sub-products:

1. Facial Recognition Module
2. Attendance Management Module
3. Analytics & Reporting Module
4. User Interface Module
5. System Integration Module

Step 2: Break Each Component into Sub-Tasks

1. Facial Recognition Module

Sub-Component	Estimated Effort (Days)
Capture facial image	2
Facial feature extraction	3
Face matching & validation	4
Error handling & accuracy tuning	2
Subtotal	**11 days**

2. Attendance Management Module

Sub-Component	Estimated Effort (Days)
Auto-mark attendance	2
Manual attendance override	1
Attendance status rules	2
Attendance storage logic	2
Subtotal	**7 days**

3. Analytics & Reporting Module

Sub-Component	Estimated Effort (Days)
Attendance analytics computation	3
Dashboard visualisation	3
Report export (PDF/Excel)	2
Subtotal	**8 days**

2. Work Breakdown Estimating - Example

4. User Interface Module

Sub-Component	Estimated Effort (Days)
Student check-in interface	2
Lecturer dashboard UI	3
Admin configuration screens	2
Subtotal	**7 days**

5. System Integration Module

Sub-Component	Estimated Effort (Days)
Integration with student database	2
Authentication & access control	2
System testing & integration validation	2
Subtotal	**6 days**

Step 3: Calculate Total Estimated Effort

Module	Effort (Days)
Facial Recognition	11
Attendance Management	7
Analytics & Reporting	8
User Interface	7
System Integration	6
Total Estimated Effort	**39 days**

Team Size	Duration (Approx.)
1 person	39 days
2 people	19–20 days
3 people	13 days
4 people	~10 days

2. Work Breakdown Estimating - Example

Agile / Timebox Context (Recommended Explanation)

If:

- 1 sprint = 2 weeks = **10 working days**
- Team size = **3 members**

Then:

- Team capacity per sprint = 3 × 10 = **30 person-days**
- Total effort = **39 person-days**

➡️ **System can be completed in about 1.5 sprints**



THANK YOU

UTM
UNIVERSITI TEKNOLOGI MALAYSIA

In the Name of God for Mankind

www.utm.my